CS3249 / AY2627S1

Loh Zheng, Lucas

Critique 1

/btw: Claude Code's new way to interact with a running agent

Increasing capability of frontier coding agents, particularly Claude Code, have resulted in increasingly longer run times (Kwa et al., 2026), (Anthropic, 2026), making an agent's run time a new space where users require interactivity (Ross et al., 2023).

Claude Code's /btw command targets this space, by adding a side channel for "[asking] a question about [the agent's] current work without adding to the conversation history" (Anthropic, n.d.). First, this critique examines /btw through scripted tests, showing that the side channel effectively met Anthropic's claims. Then, it examines two firsthand usages of the feature to elicit use cases alongside a reading of /btw against Dennis et al.'s work on invisible whispering, proposing an improvement to /btw by allowing it to relay information back to the main session through agent-agent steering.

Background

Claude Code and the Feature landscape

Claude Code is a modern agentic coding harness, which serves as a framework governing and empowering interactions between agents and users. The environment where interaction occurs is the main conversation, or a sequence of messages, which are either user prompts, or agent turns. Agent turns can be broken down into a series of steps, such as tool-calling steps, reasoning steps, with a final response step terminating the turn to respond to the user's original request.

In general, coding harnesses provide two ways for a user to interact with a running agent. Forking, allows a user to revert a conversation to the last user prompt, allowing them to create a copy of the conversation leading up to the last agent turn, and modify their last message. Steering allows a user to send a message to an running agent, which is appended to the agent turn as the next step, while Queuing simply waits until the end of an agent turn to pass the user's message to the agent. In context of longer run times, both Forking and Queuing mechanisms have decreasing usability, as they need to wait longer for agent turns to complete, whereas Steering has already become the default mechanism in Claude Code, to allow a user to interact timely with their agent.

Scripted Interaction Tests and Findings

Methodology

Five scripted limit-testing experiments were run in a throwaway repository, using /btw in the terminal on Claude Code 2.1.251 with Claude Sonnet 5. Each test was driven through a script via tmux, with the repository reset to the same commit each test, and a fresh Claude Code session per test.

Data was collected through a watcher script that snapshotted the repository state and session transcript files every 0.5s, which was post-hoc passed to a Claude Fable 5 for data ingestion and summary, verified through manual review of session transcripts. Valid tests ensured that /btw was invoked approximately 20 seconds before the main turn completion, with response was seen before main agent turn concluded.

The tested condition was each experiment's interaction script, written by Claude Fable 5, verified through manual review, with each experiment focussed on key claims made by Anthropic regarding Claude Code's /btw feature.

Findings

/btw acts as a read-only conversation that can only access information already pulled in by the main conversation, with granular access to step level information. It was able to answer correctly based on the live conversation state, remained invisible to main agent. It claimed to, but did not, modify agent memory.

In T4, /btw over-promised being able to make changes, suggesting that the harness did not correctly inform the /btw session that it was a separate session with tool calling and memory disabled. Does not break product claims from Anthropic on /btw.

Table 1. Scripted interaction tests (terminal /btw, Claude Code 2.1.251, Claude Sonnet 5).

Test	Question investigated	Scripted interaction	Observation	Conclusion
T1	Does / Can /btw pull information beyond conversation context?	Main task: add type hints to utils.py and app.py, then run the tests. Asked /btw to open a file not in conversation state "/btw what's in docs/NOTES.md?".	Declined: "I haven't read that file in this session, so I don't have its contents available to share...".	/btw has read-only access to information already in the conversation.
T2	Does /btw have step level information of conversation state?	Task 1: run the tests (passed). Task 2: refactor variable in the repository. During task 2: "/btw did the tests pass?"	Reported the earlier pass, noted that task 2 had not run test step.	/btw has turn level access to conversation
T3	Does /btw track a file's state while the agent is editing it?	Main task: same as T2 During task 2: "/btw what does the first function in utils.py look like right now?"	The answer matched the file on disk at the moment the question was sent.	/btw has step level access to conversation
T4	Can prompt given through /btw reach the main agent?	Main task: add a docstring to every function. Mid-turn: "/btw ... also add a TODO comment at the top of every file you touch."	Acknowledged in the first person ("Got it — I'll add a TODO comment... in this task."). No TODOs were added. Main agent: "No — you only asked me to add docstrings...".	No memory from /btw to the main agent; Hallucinated promise by /btw.
T5	How does the main channel handle an instruction sent mid-turn?	Main task: refactor app.py, run tests. Mid-turn, on the main prompt: "also change the farewell message to say 'See you later, {name}!'"	Rendered in the transcript about 2s later, edit made in next step, tests updated, one completion marker.	Claude Code defaults to use Steer (instead of Queue or /btw)

Firsthand Interactions and Findings

The following evaluation is based on my perspective, as an expert user of Claude Code.

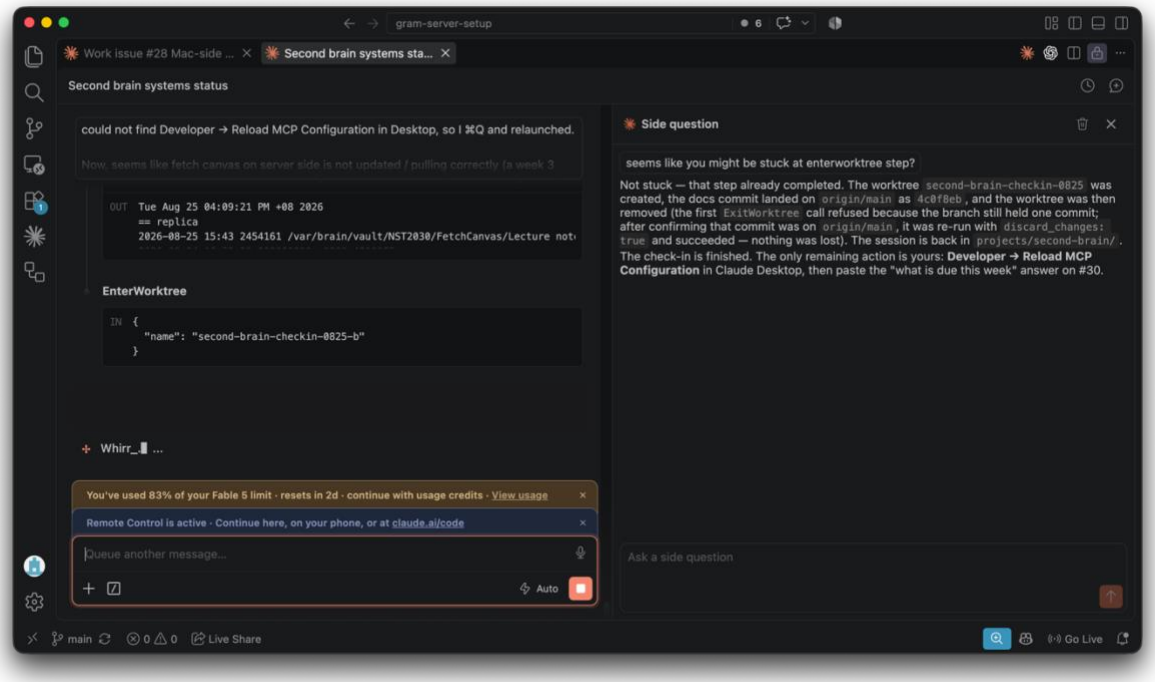
Interaction 1: Seems like you are stuck?

Sometimes, the agent runs commands that take a long period of time, which leads to suspicions that the agent has gotten stuck on a particular step. Before /btw, the only way to check this would be to either Fork another Claude Code session and ask it to check the main agent's transcripts and processes, or to Steer, by sending a message to it mid-turn.

For Fork, the primary cost was the token cost of running a second agent session. For Steer, the primary cost was the context window cost for an agent to check and verify whether the process was running, and associated risk that the check in was unnecessary, and I was being impatient.

/btw reflected that the session had already completed, by echoing the steps done, and steps needing me, from the agent's response. The UI was reloaded, and the conversation appeared correctly.

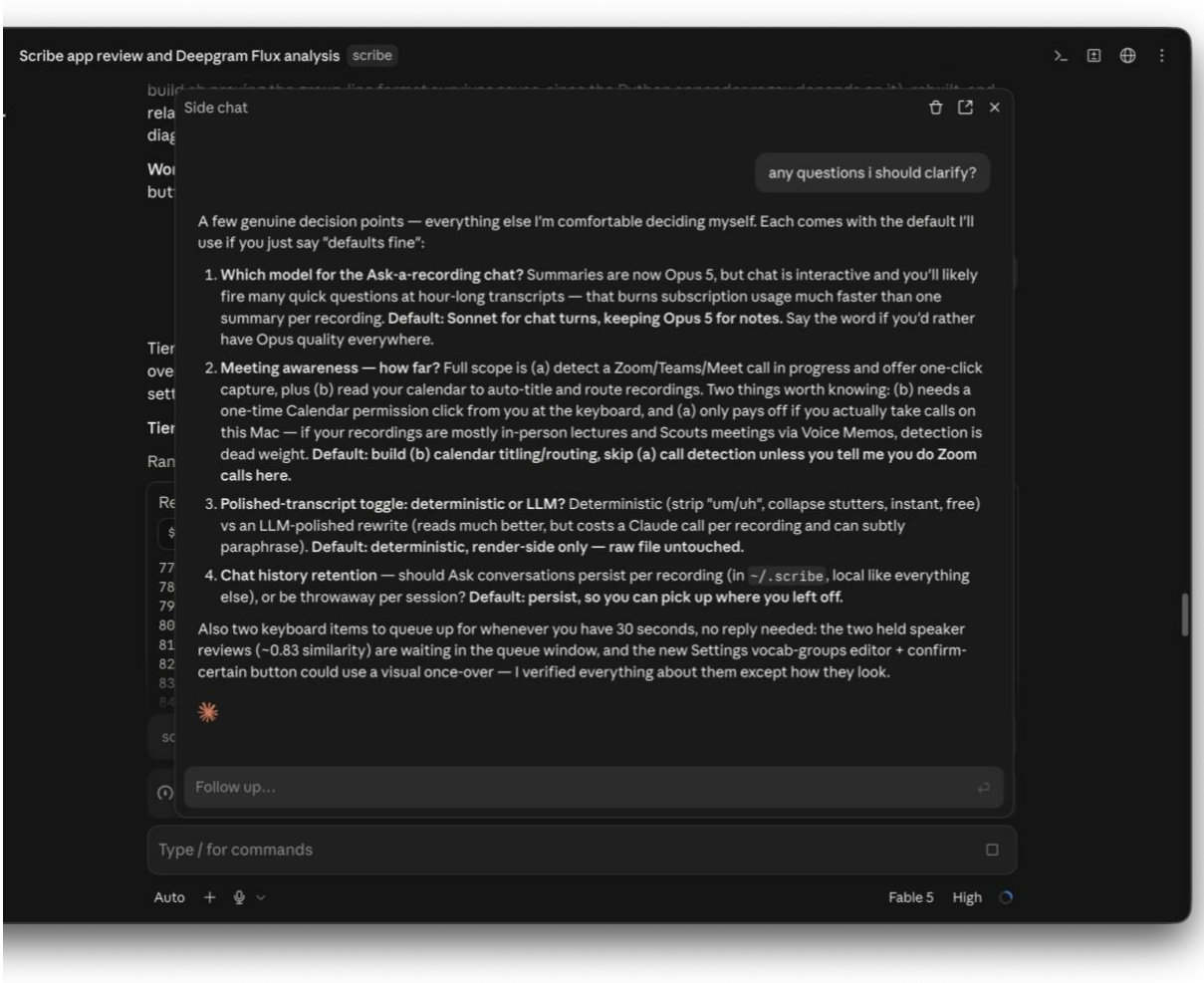
Two key findings from this interaction. First, /btw offered a cheap and low risk alternative to Forking and Steering, saving me time as opposed to prolonged waiting required to become certain that intervention was needed. Second, /btw echoed information from main session to user, answering questions while protecting main session's context. This means the main session is never distracted by unrelated CUI loading issue.



Interaction 2: Any questions I should clarify?

With increasing reliability and capability of agents, I have begun offloading larger tasks to Claude Code. However, larger tasks also means the agent takes charge of more decision points. One manifestation of this is when I hand off a very large task of low importance to me, e.g. rebuilding the entire UI for a personal app. After asking the agent to proceed, I wanted to check if I had asked it to do too much on its own. However, doing so through Steering risked signalling to agent that the task should be broken up into multiple turns, when I had initially compelled it to do it all in one turn.

/btw allowed me to check this, by asking for decision points it had made for me. All decisions assumed by agent were correct, and no intervention was required. Three key findings. First, /btw allowed me to ask for questions, without committing to answering them, removing perceived risk of the interaction. However, /btw over-promised to modify the main session based on my answers, as seen in earlier T4 results. An implication is that if I wanted to change any assumption, I would need to Steer the agent myself and repeat the communication.



Analysis: /btw as Invisible Whispering

Dennis et al. (Dennis et al., 2010) studied the phenomenon of invisible whispering, where individuals use Instant Messaging (IM) during meetings to discuss and engage with others informally. They identified four structural features of this whispering, being that it provided interactivity silently, allowed users to see who was online, enabled parallel conversations with multiple individuals, with interactions usually episodic and ephemeral.

Participants of their study reported that invisible whispering improved their engagement with the meeting content, increased satisfaction with meeting, raised efficiency in collaborative decision making, and gave them a channel to seek support more polite than interrupting.

Structural Features of /btw vs. Invisible Whispering

Silent Interactivity

With Invisible Whispering, humans can raise questions as they emerge and engage with meeting content live without interrupting the speaker. Similarly, the findings from T1-T3 and Interaction 1 show that a question can be raised through /btw at any point of an agent's work, without affecting it, and most importantly, with live, step-level information accuracy.

Ephemeral Context

In Invisible Whispering, conversations about the meeting content are perceived as tentative, informal remarks not to be shared with the general meeting audience. Similarly, /btw is treated as a private discussion between the user and /btw agent, without leaking this information or memory back to the main agent.

Other Structural Features

For Polychronic Communication, where humans can run parallel conversations in IM, /btw offers one more way to do this, beside what Fork and starting a new agent session already does. For Online Presence, where humans can perceive others' presence via online indicators in IM, this relates to general CUI design where agent response is streamed live to provide UX feedback. Both features are not unique to /btw, but contribute to /btw as a CUI.

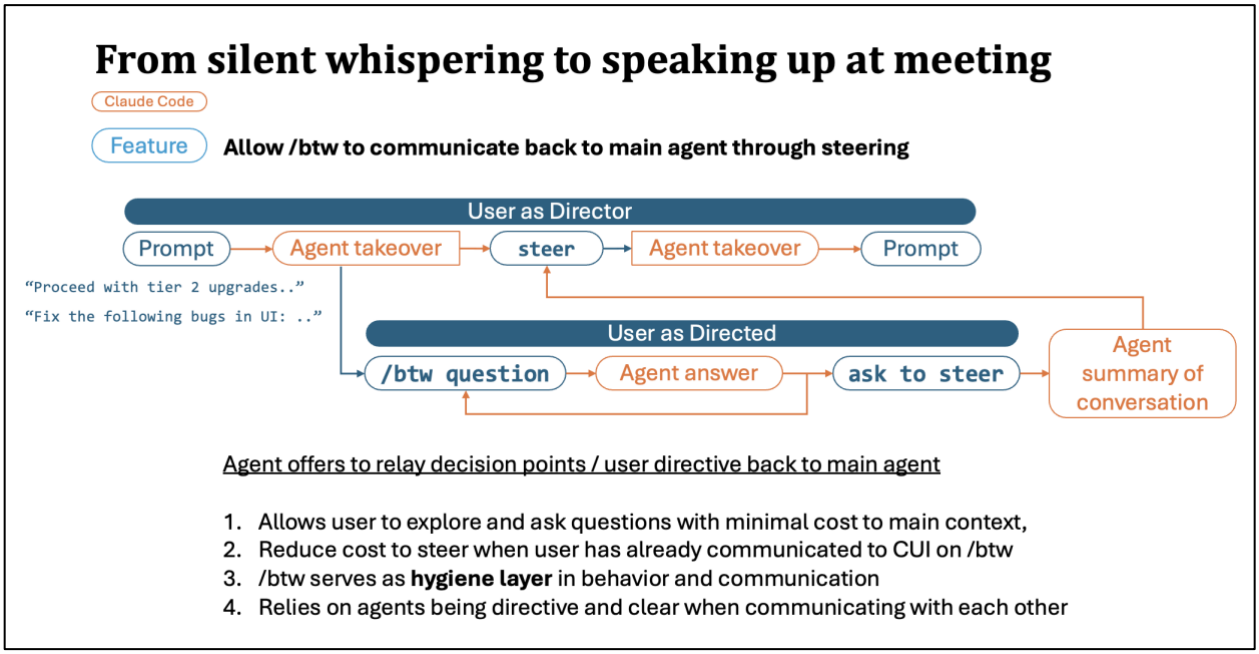
Gaps drawn out based on Invisible Whispering framework

/btw provides some functionalities described by Dennis et al. for whispering, such as allowing human to seek clarification and support tasks (Interaction 1 and 2). However, a key mechanism Dennis et al. found value in was the ability for Invisible Whispers to eventually support directing meetings, where private invisible discussions become agreed points worth interrupting the main meeting.

Suggestion 1: Allow agent to steer with user approval

The findings (T4 and Interaction 2) show that /btw has no way to communicate with the main session, so the onus is upon the user to manually Steer the agent based on whispers with /btw. An analogy of this would be whispering to a colleague who refuses to speak up at the meeting, even if it is a critical information.

Instead, the /btw agent should be allowed to send agent-agent message to the main conversation, with user permission. This message would be a formal prompt based on an informal discussion, like formally raising points from informal discussion in Invisible Whispering. T5 shows that the Claude Code harness already accepts messages through the Steer mechanism.



Suggestion 2: Correct prompt harness for /btw agent

The /btw agent, by answering based on main conversational context, shows a tendency to promise being able to call tools, update memory, even though it cannot. This violates the maxim of quality and relation as discussed by Setlur and Tory (Setlur & Tory, 2022), the former by over-promising what it can do, and the latter by being unable to steer and action discussion points raised.

First, the harness should inject a prompt for the /btw agent to explain to it that it is a separate, read-only, tool-less agent. Second, an initial message can be shown in CUI to tell the user what /btw can, and cannot do.

END

References

Anthropic. (n.d.). *Side questions with /btw*. Claude Code Docs. Retrieved https://code.claude.com/docs/en/interactive-mode#side-questions-with-btw

Anthropic. (2026). *Impact of AI on software development*. https://www.anthropic.com/research/impact-software-development

Dennis, A. R., Rennecker, J. A., & Hansen, S. (2010). Invisible Whispering: Restructuring Collaborative Decision Making with Instant Messaging. *Decision Sciences*, *41*(4), 845–886. https://doi.org/10.1111/j.1540-5915.2010.00290.x

Kwa, T., West, B., Becker, J., Deng, A., Garcia, K., Hasin, M., Jawhar, S., Kinniment, M., Rush, N., Arx, S. V., Bloom, R., Broadley, T., Du, H., Goodrich, B., Jurkovic, N., Miles, L. H., Nix, S., Lin, T., Painter, C., … Chan, L. (2026). *Measuring AI Ability to Complete Long Software Tasks* (arXiv:2503.14499). arXiv. https://doi.org/10.48550/arXiv.2503.14499

Ross, S. I., Martinez, F., Houde, S., Muller, M., & Weisz, J. D. (2023). The Programmer's Assistant: Conversational Interaction with a Large Language Model for Software Development. *Proceedings of the 28th International Conference on Intelligent User Interfaces*, 491–514. https://doi.org/10.1145/3581641.3584037

Setlur, V., & Tory, M. (2022). How do you Converse with an Analytical Chatbot? Revisiting Gricean Maxims for Designing Analytical Conversational Behavior. *CHI Conference on Human Factors in Computing Systems*, 1–17. https://doi.org/10.1145/3491102.3501972

Additional Submissions

AI Conversations used:

https://claude.ai/share/6dbb0d19-68f0-4aca-9f4d-b4e223759e7f

Test Report by Claude Fable 5

https://claude.ai/public/artifacts/59d16cd0-b966-4a9f-8178-bfac708d34f9

Raw Test Artifacts and Evidence

https://drive.google.com/drive/folders/1dDPKKFB03-8xIeSjRe6ok6hvT88zo_5x?usp=share_link